Knowledge Center

What Is the SSDLC (Secure Software Development Life Cycle)?

The Secure Software Development Life Cycle (SSDLC) is a framework for developing secure software. It is a set of processes and activities that organizations follow to ensure that their software is developed with security in mind.

The goal of the SSDLC is to identify and mitigate potential security vulnerabilities and threats in the software development process, so that the final product is as secure as possible. The SSDLC typically includes activities such as threat modeling, secure coding practices, security testing, and security reviews.

This is part of a series of articles about DevSecOps.

Why Is SSDLC Important?

The SSDLC is important because it helps organizations develop secure software that is resistant to attacks and can protect sensitive data. In today's digital landscape, software security is more important than ever, as software is used to store and process vast amounts of sensitive information. If software is not developed with security in mind, it can be vulnerable to attacks that can compromise sensitive data and cause harm to individuals and organizations.

By following the SSDLC, organizations can ensure that their software is developed with security considerations at every stage of the development process. This helps to identify and mitigate potential vulnerabilities and threats before they become a problem, and it can help organizations build a reputation for developing secure software. Additionally, SSDLC can help comply with industry regulations and standards that require secure software development practices.

Embedding Security into All Phases of the SSDLC

Here is an overview of how security can be embedded into each phase of the SSDLC:

Planning

During the planning phase, it is important to identify the security requirements for the software and incorporate them into the project plan. This may include conducting a risk assessment to identify potential security threats and vulnerabilities and determining the appropriate controls to mitigate them.

Requirements and Analysis

In this phase, it is important to clearly define the security requirements for the software and ensure that they are understood and included in the software design. This may include developing a threat model to identify potential attacks and determining how the software will protect against them.

Design and Prototyping

During the design and prototyping phase, it is important to design the software with security in mind. This may include using secure design patterns, implementing appropriate controls, and conducting a security review of the design.

Development

In the development phase, it is important to follow secure coding practices to ensure that the software is developed with security in mind. This may include using static analysis tools to identify potential vulnerabilities, following secure coding standards, and conducting regular code reviews.

Deployment

During the deployment phase, it is important to follow secure deployment practices to ensure that the software is deployed securely. This may include conducting a security assessment of the deployment environment, implementing appropriate controls, and following best practices for securing the software in production.

Maintenance

In the maintenance phase, it is important to continue to prioritize security. This may include regularly patching the software to fix vulnerabilities, monitoring the software for security issues, and conducting regular security reviews to ensure that the software remains secure over time.

Best Practices to Secure the SDLC

Prepare Your Organization

Some ways to prepare your organization for secure software development include:

- **Establishing a secure software development policy:** Having a clear policy in place that outlines the expectations and requirements for secure software development can help ensure that all team members understand the importance of security and are aware of the specific steps they need to take to ensure the security of the software.
- **Providing training and resources:** Providing training and resources to team members on secure software development best practices can help ensure that they are able to develop secure software. This may include providing training on secure coding practices, threat modeling, and other relevant topics.
- **Establishing a secure development environment:** Ensuring that the development environment is secure can help prevent security breaches during the development process. This may include implementing controls such as access controls and vulnerability management.
- **Implementing a secure change management process:** Having a secure change management process in place can help ensure that all changes to the software are reviewed and approved by appropriate parties before being implemented. This can help prevent unapproved changes from being made that could introduce vulnerabilities into the software.

Add Security Practices to Organizational Processes

Some ways to produce well-secured software include:

- **Conducting threat modeling:** Threat modeling is a process that helps identify potential attacks and determine how the software will protect against them. Conducting threat modeling during the design phase can help ensure that the software is designed with security in mind.
- **Using secure design patterns:** Using secure design patterns can help ensure that the software is developed in a way that is resistant to common attacks. For example, using patterns such as the "fail-secure" pattern can help ensure that the software fails safely in the event of an attack.
- **Implementing appropriate controls:** Implementing appropriate controls, such as access controls and input validation, can help prevent attacks and protect sensitive data.
- **Following secure coding practices:** Following secure coding practices, such as using static analysis tools and following coding standards, can help ensure that the software is developed with security in mind.

- **Conducting regular code reviews:** Conducting regular code reviews can help identify potential vulnerabilities and ensure that the code is of high quality.

Proactively Assess and Verify Security

Some ways to protect your software include:

- **Conducting a security assessment of the deployment environment:** Before deploying the software, it is important to assess the security of the deployment environment to ensure that it is secure. This may include identifying potential vulnerabilities and implementing controls to mitigate them.
- **Patching the software regularly:** Regularly patching the software to fix vulnerabilities is important for ensuring the security of the software and preventing breaches. Keeping software up to date to fix known vulnerabilities will reduce organizational risk.
- **Monitoring the software for security issues:** Regularly monitoring the software for security issues, such as unusual activity or attempted attacks, can help identify potential issues and allow for timely response.
- **Conducting regular security reviews:** Conducting regular security reviews of the software can help ensure that it remains secure over time. This may include reviewing the security of the software's architecture and design, as well as its implementation and deployment.

Respond to Vulnerabilities

Some steps for responding to vulnerabilities include:

- **Establishing a process for identifying vulnerabilities:** Having a process in place for identifying vulnerabilities, such as using static analysis tools or conducting regular code reviews, can help ensure that vulnerabilities are identified and addressed promptly.
- **Assessing the impact of vulnerabilities:** Once a vulnerability has been identified, it is important to assess the impact it could have on the software and the data it processes. This will help determine the appropriate response to the vulnerability.
- **Prioritizing the response to vulnerabilities:** It is important to prioritize the response to vulnerabilities based on the potential impact they could have. Higher-impact vulnerabilities should be addressed as a priority.
- **Developing a plan to address vulnerabilities:** Once the impact of a vulnerability has been assessed, a plan should be developed to address it. This may include implementing a patch or other corrective action to fix the vulnerability.

Communicating with stakeholders: It is important to communicate with relevant stakeholders, such as customers and users, about vulnerabilities and the actions being taken

to address them. This can help build trust and ensure that any necessary actions are taken in a timely manner.

DevSecOps with HackerOne

HackerOne helps organizations accelerate the journey to DevSecOps by combining the expertise of the world's largest community of ethical hackers with a seamless platform that orchestrates workflows, testing programs, scoping and reports.

HackerOne bridges requirements across development and security teams that:

- Helps application development teams deliver software with fewer flaws
- Fosters collaboration and trust between security and development teams
- Extends the reach of security by educating developers to look for ways to reduce vulnerabilities
- Optimizes security practices for the scale and speed needed for modern application development and deployment
- Improves adherence to compliance, legal and regulatory requirements

With a faster path to DevSecOps with solutions from HackerOne, organizations will release applications with a greater resistance to attack while maintaining the speed of their DevOps pipeline.

Learn more about the HackerOne Security Platform

Contacted by a hacker?

Cybersecurity

Attacks

DevSecOps
